Master 2 CIM - DA - DT 2026/2027
ICOM - Université de Lyon 2

Module Champ d'Application (Angular JS)

Travaux Pratiques 2

Gestion des données dynamiques (suite et fin) et structures de documents

---

Objectif: Dans ce TP, nous allons terminer d'explorer les données dynamiques à travers les liaisons à double sens et en exploitant les formulaires (`FormsModule`). Ensuite, nous montrerons les possibilités qu'offrent *Angular* au niveau de la structuration des documents.

Partie I: Suite et fin données dynamiques

Pour terminer la partie données dynamiques, nous allons illustrer le ***Two ways Binding***. Ce mécanisme permet d'échanger des données dans les deux sens, c'est à dire à la fois de HTML vers *TypeScript* et réciproquement de *TypeScript* vers HTML.

Dans cette séance, vous pourrez utiliser comme base soit votre projet du TP précédant soit partir de la correction accessible sur *moodle* à l'adresse suivante :

`https://moodle.univ-lyon2.fr/course/section.php?id=194008`

1.1 Le projet récupéré ne contient pas le répertoire *node_module* mais ce dernier sera facilement créé à partir de la commande suivante :

```
cd TP2AngularBase
npm install
ng serve --open
```

La double liaison peut s'effectuer en utilisant le module `FormsModule` que vous pouvez importer dans votre application.

1.2 L'importation d'un module dans votre application s'effectue directement dans le composant, donc dans ce TP dans `app.ts` :

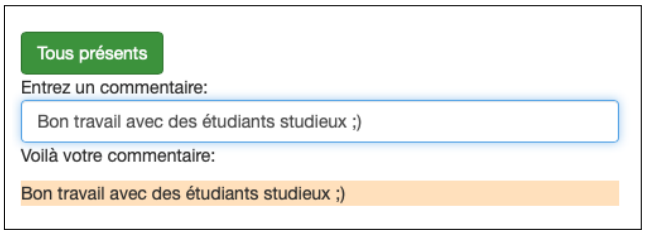


en rajoutant un *import* comme sur l'image ci-dessus et en l'ajoutant dans les `imports` du composant. La ligne 5 sera alors dégrisée.

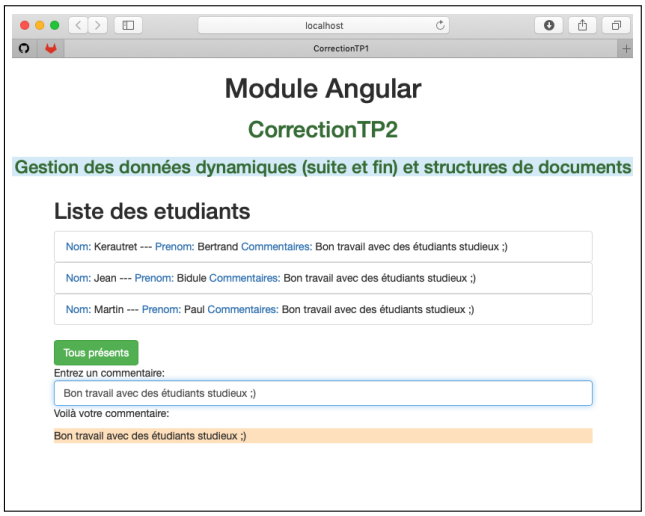
1.3 Pour illustrer cette liaison, il s'agit de rajouter un champ `<input>` et de lier son entrée à une variable. Cette double liaison est définie à travers les symboles `[()]` et en utilisant la directive `ngModel`. Pour tester la fonctionnalité, après le bouton *Tous présents*, rajoutez le champ *input* suivant :

```
<input type="text" class="form-control" [(ngModel)]="commentaireSeance">
```

Définissez ensuite la variable associée `commentaireSeance` puis testez d'utiliser la variable en exploitant une liaison par *string interpolation*. Vous obtiendrez par exemple le test suivant :



1.4 Pour compléter cette double liaison, on souhaite que la variable soit aussi exploitée dans les différentes fiches d'étudiants. Modifier le composant des étudiants de façon à ce que le commentaire entré par l'utilisateur apparaisse dans toutes les fiches comme sur l'image suivante :



Indication : Il s'agit d'utiliser la liaison par propriétés personnalisées.

Partie II: Directives structurelles et contrôle de flux

Après avoir vu les différents mécanismes de liaisons dynamiques, il est aussi utile de vouloir modifier la structure de votre application à partir de nouvelles directives. Même s'il est possible de rajouter ses propres directives, nous allons mettre en pratiques les deux directives suivantes :

- directive conditionnelle (***ngIf***) : cette directive permet d'afficher un composant uniquement si la condition affectée à ce mot clé est vraie. Par exemple, si on rajoute `*ngIf="condition"`, alors le composant ne sera affiché que si la condition est vraie.
- directive itérative (***ngFor***) : elle permet de répéter l'affichage d'un bloque.

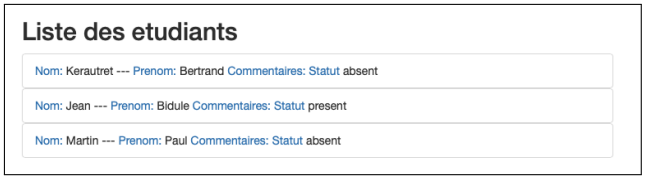
Remarque Angular récent : Angular propose désormais une syntaxe de contrôle de flux intégrée avec @if et @for. Dans ce TP, nous utilisons volontairement *ngIf et *ngFor afin de comprendre les directives structurelles, qui restent très présentes dans les projets Angular existants. Pour continuer à tester ***ngIf*** et ***ngFor***, il faut les importer :

```
import { NgIf } from '@angular/common';
import { NgFor } from '@angular/common';

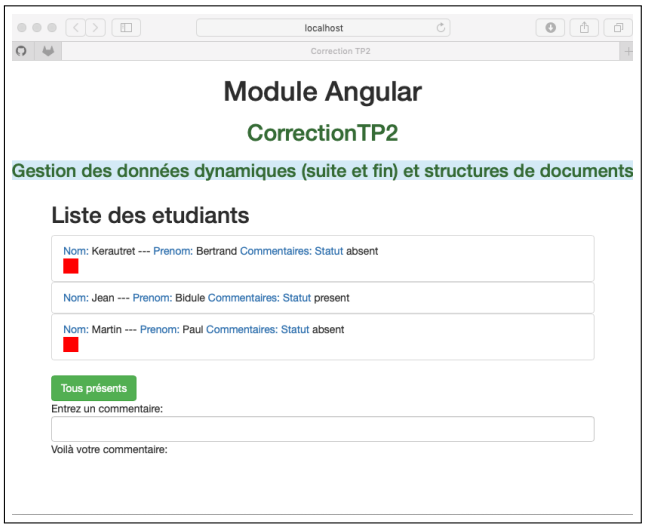
@Component({
  imports: [RouterOutlet, Etudiant, FormsModule, NgIf, NgFor],
```

Avant de tester la directive conditionnelle, nous allons tout d'abord rajouter l'affichage et la configuration du statut (présent/absent) d'un étudiant.

2.1 Modifier les différents fichiers de façon à obtenir l'affichage ci-dessous :



2.2 Dans le fichier template de l'étudiant, utilisez la structure conditionnelle de façon à afficher un carré rouge sur chaque étudiant absent. L'affichage pourra ressembler au suivant :



L'autre directive structurelle importante est la boucle **ngFor* qui va permettre de répéter la génération de différents éléments. La syntaxe utilise le mot clé (`*ngFor=" let x of <elem>"`) où `elem` va représenter un container pouvant être itéré comme par exemple un tableau et `x` sera la variable utilisable à droite d'un attribut dont le nom sera mis entre crochet. La boucle générera autant de composant qu'en contient `elem`.

2.3 Dans le fichier `app.component.ts`, rajoutez un tableau `tabEtudiant` qui contiendra chaque étudiant. La syntaxe d'un tableau d'objet est la suivante :

```
tabEtudiant = [
    { nom: 'Dupont', prenom: 'Jean', present: true },
    { nom: 'Martin', prenom: 'Sophie', present: false }
];
```

Chaque tableau associatif aura ainsi les champs associés aux nom, prénom, statut. Par exemple `tabEtudiant[0].attribut1` donnera String1.

2.4 En revenant dans le fichier template `app.component.html`, utilisez une boucle pour afficher tous les étudiants contenus dans le tableau précédant `tabEtudiant`. Vérifiez que l'affichage fonctionne bien en rajoutant quelques nouveaux étudiants dans le tableau.

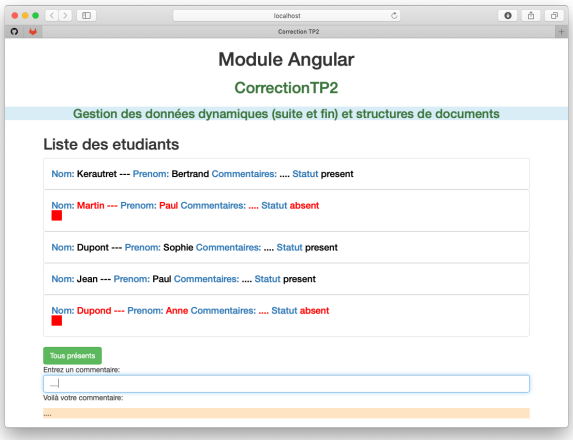
Partie III: Directives par attributs

Une autre façon de changer le DOM du document est d'utiliser des directives par attributs. En fait, nous avons utilisé ces directives dans la première partie de ce TP avec la directive `ngModel`. Dans la suite nous allons explorer d'autres directives avec `ngStyle` et `ngClass`.

La directive `ngStyle` permet de changer de manière dynamique un style défini dans un composant. La syntaxe est assez simple puisqu'il s'agit de mettre la directive entre crochet et de l'affecter à un ensemble `clé: valeur` où la valeur sera interprétée en *TypeScript*. Par exemple, `[ngStyle]="{color: 'red'}"` produira un style basé sur la couleur rouge qui s'appliquera tout le temps.

3.1 Dans le fichier `etudiant.component.ts` rajoutez une fonction `getColorEtu()` qui renverra une couleur rouge si l'étudiant est absent et une couleur noir sinon.

3.2 En revenant dans le fichier `etudiant.component.html`, appliquez une balise `h4` au contenu de l'item représentant l'étudiant et appliquer la directive `ngStyle` décrite précédemment.

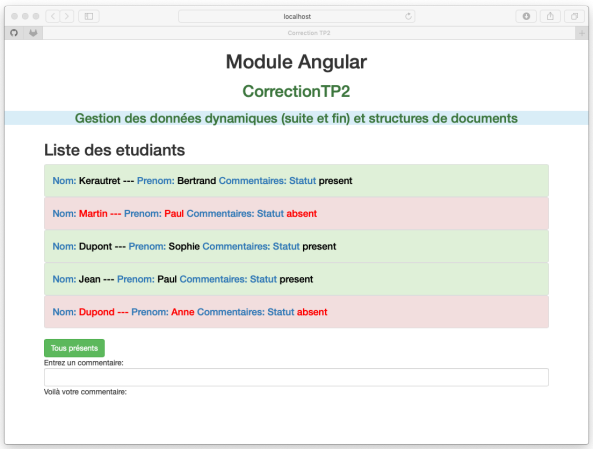


Une directive très utile est la directive `ngClass` qui fonctionne sur le même principe que pour la directive précédente. Dans la suite il s'agit de l'appliquer pour changer la classe d'une représentation d'un étudiant (composant `li`).

3.3 En utilisant les style suivant, utilisez la directive pour avoir des items différencié suivant les la présence ou nom des étudiants.

- `'list-group-item'` : style par défaut.
- `'list-group-item-success'` : style pour étudiant présent.
- `'list-group-item-danger'` : style pour étudiant absent.

3.4 Améliorer l'affichage de façon à avoir l'affichage suivant :



Remarque Angular récent : Il est désormais possible dehcnger directement la couleur d'un style en faisant :

```
<h4 [style.color]="getColorEtu()">
```

```
<li [class.list-group-item-danger]="!present">
```

L'objectif des questions suivantes est de rajouter un bouton permettant de changer le statut d'un étudiant.

3.5 Dans le fichier `etudiant.component.html`, rajoutez deux boutons *absent* et *présent* qui permettront plus tard de changer le statut de l'étudiant.

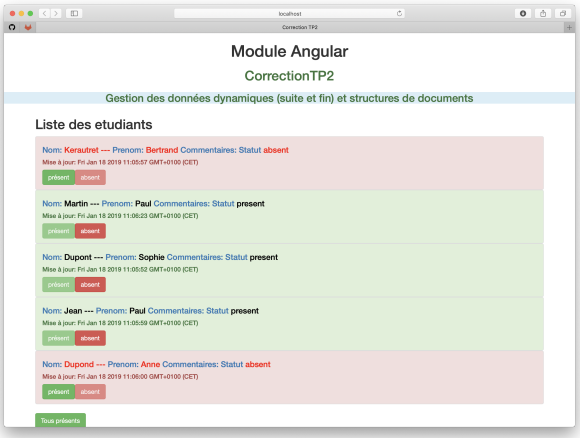
3.6 Implémentez les fonctions de façon à ce que ces boutons permettent de changer le statut de l'étudiant.

3.7 Améliorez l'interface de façon à ce que les boutons soient désactivés suivant le statut de l'étudiant.

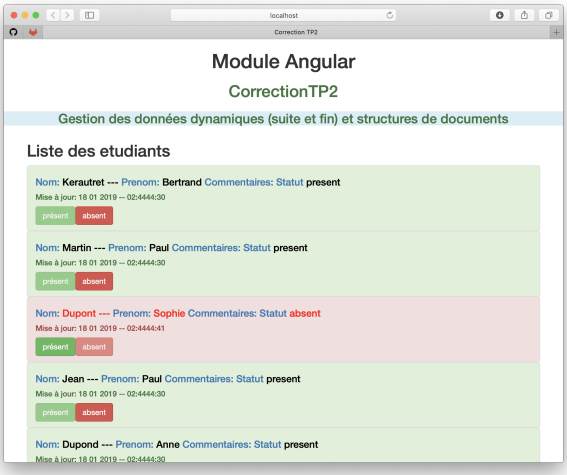
Partie IV: Pour ceux qui ont fini

Un autre outil très utile en *TypeScript* est l'utilisation des *pipe* qui permettent de modifier les données en temps réel. Comme première utilisation nous allons utiliser le *pipe* avec la fonction date.

4.1 Dans le fichier `etudiant.component.ts`, rajoutez un nouvel attribut de classe (`derniereModif`) qui stockera la date de dernière modification. **Indication : créer un nouvel objet Date** et mettra à jour la date à chaque changement de statut de l'absence.



4.2 Le *pipe* s'utilise simplement en plaçant le symbole | entre deux fonctions/expression. Exploitez le *pipe* avec votre variable `derniereModif` et la fonction date. Changer le format de façon à avoir un affichage :



4.3 Pour ceux qui ont fini, rajouter la gestion d'un compteur pour compter le nombre de fois où l'étudiant a été compté absent. Vous pourrez utiliser un *timer* pour re activer un bouton *absent*.

4.4 Implémentez la fonction restante associé au bouton *Tous Absents*. Comme vous pouvez le voir, la fonction n'a plus d'effet dès lors que l'utilisateur à changé le statut d'un étudiant. Cela peut s'expliquer par le fait que la liaison dynamique entre les valeurs du tableaux et la variable représentant le statu de l'étudiant a été supprimé par l'action du bouton d'absence ou présence.

Nous verrons dans les prochaines séances comment garder une seule instance des données et comment contrôler les valeurs avec la notion de services.